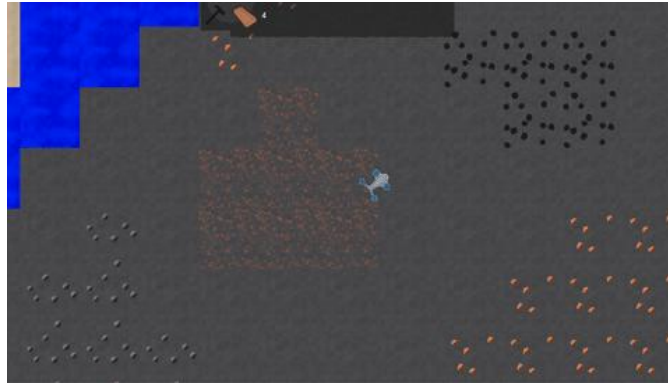
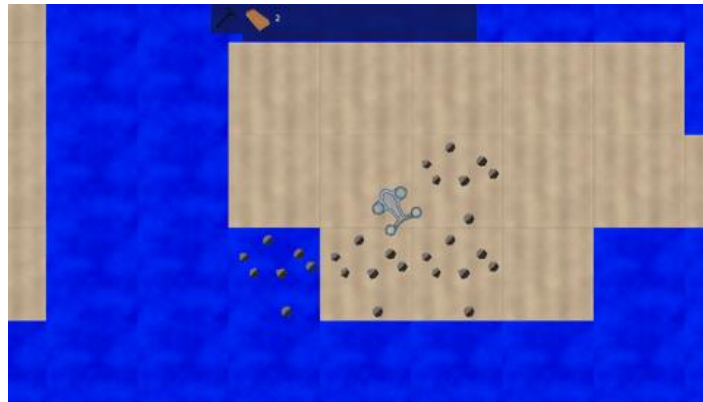
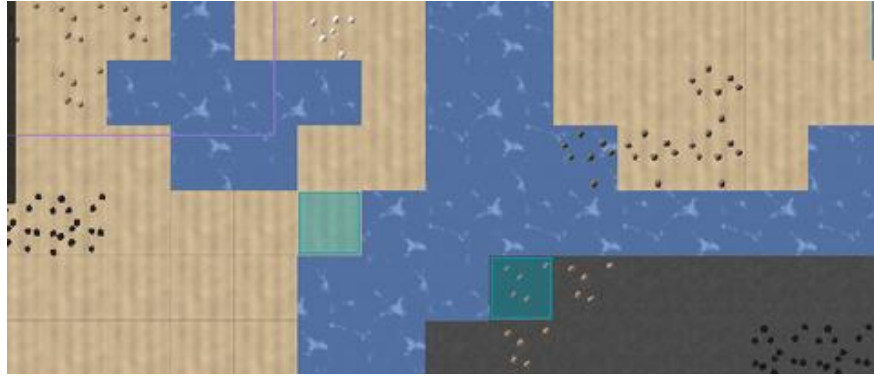


**(GAME DEVELOPMENT – ISSUE IDENTIFICATION)**

In the initial version of the camera controller script, the zoom responded directly and immediately to input, which made the camera feel choppy and almost jittery in motion, as shown above. During scrolling to zoom out the camera snapped the zoom value to a new level in a single frame, so there was no easing to soften the transition. Because of this, even small adjustments felt harsh and disorienting, especially when the player tried to track movement or aim precisely. This also made the game feel less polished overall, because the camera felt like a debug tool rather than something tuned for gameplay. In short, the instantaneous zoom change was very jarring, and made the game feel unfinished.

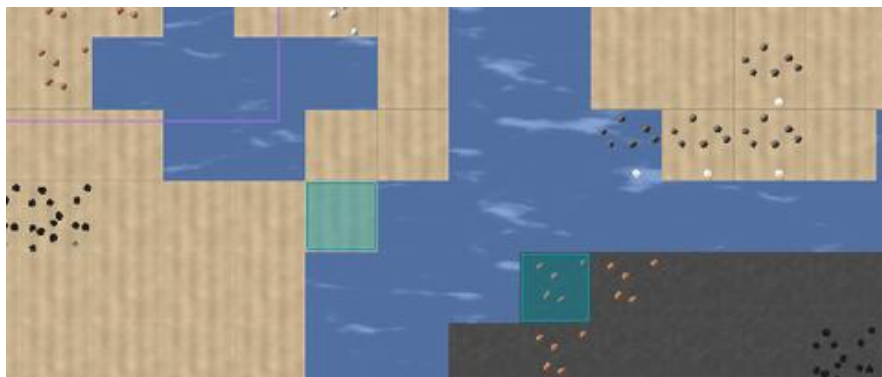
**(MATH & GAME DEVELOPMENT – PROCESS TO FIX & RESULT)**

In the updated version, I treated the current zoom like a value that should approach a target over time instead of jumping there instantly, which is what you see above. I did this by adding a `current_zoom` variable as well as a `target_zoom` variable, then used `current_zoom = current_zoom.lerp(target_zoom, 1.0 - exp(-delta * zoom_speed))` to create a smooth exponential easing effect each frame. The term `1.0 - exp(-delta * zoom_speed)` effectively gives me a time-based interpolation factor. Which then is scaled by `zoom_speed` so I can control how quickly the zoom converges to the target. This combination of exponential smoothing, and separating target from current state makes the zoom transitions feel deliberate, fluid, and soft, which significantly improves the game feel and gives the camera a much more polished, professional quality.



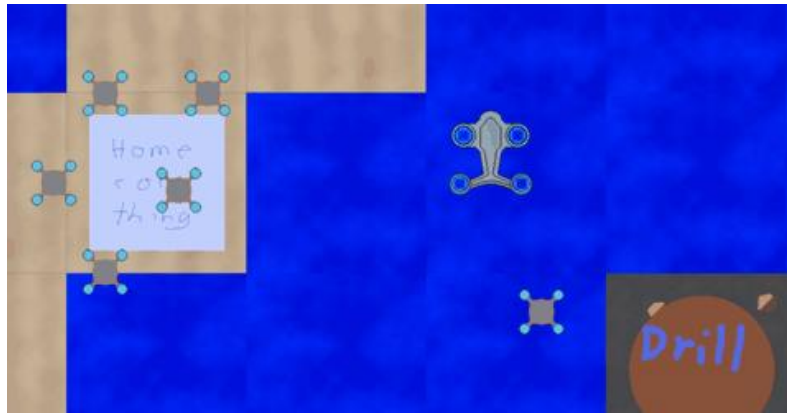
(GAME DEVELOPMENT – ISSUE IDENTIFICATION)

In the initial version of the water shader (shown above), it had a huge issue where when zooming it would zoom itself, which was a really weird issue. It also seemed to act very oddly when set to tile as the shader is being applied to a tileset used in a tilemap.



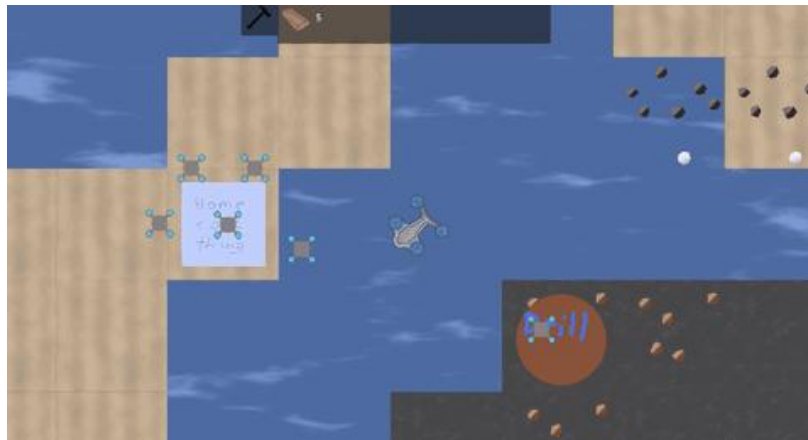
(MATH & GAME DEVELOPMENT – PROCESS TO FIX & RESULT)

The fix took me a very long time to find out, but it turned out to be an issue specifically regarding the `TEXTURE_PIXEL_SIZE`, which was crucial as without it the UV was scaling in camera space, so once I added that, we were then able to properly have it work independent from the camera.



(GAME DEVELOPMENT – ISSUE IDENTIFICATION)

In the initial version of the drone movement system, the drones would snap to full speed the instant they received a movement command, which made them feel very unnatural. They also lacked any sense of momentum or weight, since they instantly would go to max speed when getting a task assigned to them. On top of that, the drones were completely static when hovering, which made them look more like sliding sprites than actual flying objects. It's especially bad once they delivered to their location and stopped moving as it looked like they had died.



(MATH & GAME DEVELOPMENT – PROCESS TO FIX & RESULT)

My solution was to implement two separate systems working together to sell the illusion of realistic drone flight. For the acceleration, I introduced a `current_speed` variable that's separate from `max_speed`, then made acceleration and deceleration available in engine for configuration. In the movement logic, I calculated a `desired_speed` based on distance to target using a `slowdown_factor` (which is my deceleration value) and for acceleration I used `current_speed = move_toward(current_speed, desired_speed, acceleration * delta)` when speeding up. For the hovering effect, I separated the drone's visual position from its actual movement position by introducing a `base_position` that handles all the pathfinding math, then added a bobbing system with `bobbing_time += delta * bobbing_speed` and a sine wave offset calculated as `sin(bobbing_time) * bobbing_amplitude` applied only to the y-axis. The visual position becomes `global_position = base_position + Vector2(0, bobbing_offset)`, which means the drone's world position moves smoothly through space while the rendered sprite gently bobs up and down continuously.